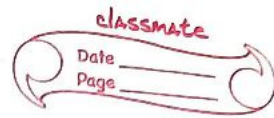


C++ NOTES



Introduction to C++

- * C++ is an object oriented programming language. It was developed by Bjarne Stroustrup at AT&T Bell's laboratory in 1979 at Murray Hill, New Jersey, USA.
- * Initially it was called "C with classes". However in 1983 the name was changed to C++.
- * Supports procedural, object oriented and generic programming.
- * Used for system software, games, competitive programming and applications.

Basic structure of C program.

#include <iostream> → Preprocessor Directives.
using namespace std; → Name space declaration.

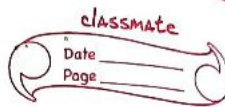
int main() → main() function
{

cout << "Hello World"; → Program statement and expression
return 0; → return statement
}

- * C++ is a superset of C i.e. all C programs are also C++ programs.

C++ program to add two numbers.

```
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    cin >> a >> b;
    cout << "Sum=" << a+b;
    return 0;
}
```



C++ program to find largest of three numbers.

```
#include <iostream>
using namespace std;
int main() {
    int a, b, c;
    cout << "Enter three numbers;";
    cin >> a >> b >> c;
    if (a > b && a > c)
        cout << "largest no is=" << a;
    else if (b > c)
        cout << "largest no is=" << b;
    else
        cout << "largest no is=" << c;
    return 0;
}
```

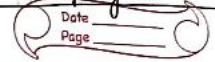
C++ program to find factorial of a Number.

```
#include <iostream>
using namespace std;
int main() {
    int n;
    long long fact=1;
```

```
    cout << fact;
    return 0;
}
```

Program to take address as input and display it as output

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string address;
    cout << "Enter your address? ";
    getline(cin, address); // getline used becoz address may
    cout << "Your address is:" << address; // contain spaces.
    return 0;
}
```



Program to take phone number as input and Count frequency of each number.

```
#include <iostream>
using namespace std;
int main() {
    long long n; int f[10]={0};
    cin >> n;
    while(n)
    {
        f[n%10]++;
        n/=10;
    }
    for (int i=0; i<10; i++)
        if (f[i]) cout << i << ":" << f[i] << " ";
    return 0;
}
```

Input: 9876678901

Output: 0:1 1:1 6:2 7:1 8:2 9:2

Program to demonstrate class and objects.

classmate
Date _____
Page _____

```
#include <iostream>
using namespace std;
class student {
private: // can't be access outside the class.
    int rollno;
    string name;
public: // use to change value in the program.
    void initialize() {
        cout << "enter rollno: ";
        cin >> rollno;
        cout << endl << "enter name: ";
        cin >> name;
    }
    void display() {
        cout << "rollno = " << rollno << endl << "Name = " << name;
    }
};

int main() {
    student st;
    st.initialize();
    st.display(); // (changing) st.rollno = 200;
    return 0;
}
```

Static Variable

```
#include <iostream>
using namespace std;
void counter() {
    static int count = 0; // hold value in function call.
    int var = 0;
    count++;
    var++;
    cout << "Count = " << count << endl;
    cout << "Var = " << var << endl;
}
```

```
{
int main() {
    counter();
    counter();
    counter();
    counter();
    return 0;
}
```

Output:

Count=1
Var=1
Count=2
Var=1
Count=3
Var=1
Count=4
Var=1

classmate
Date _____
Page _____

```
#include <iostream>
using namespace std;
class MyClass {
private:
    static int objCount;
public:
    MyClass() {
        objCount++;
    }
    static int getObjCount() {
        return objCount;
    }
};

int MyClass::objCount = 0;
int main() {
    MyClass obj1;
    MyClass obj2;
    MyClass obj3;
    cout << "Number of objects: " << MyClass::getObjCount() << endl;
    return 0;
}
```

OOP'S (Object oriented Programming)

It is a programming paradigm based on the concept of objects, which encapsulate data and behaviors (functions) together.

Principles of OOPS

- (i) Encapsulation: Wrapping data and functions into a single unit (class).
- (ii) Abstraction: Hiding implementation details and showing only essential features.
- (iii) Inheritance: Mechanism of deriving new classes from existing ones.
- (iv) Polymorphism: Ability of functions or operators to behave differently based on context.

Advantages of OOPS

- (i) Promotes code reusability.
- (ii) Enhances modularity and maintainability.
- (iii) Provides better data security.
- (iv) Models real-world problems effectively.

Features of C++

- (i) Object oriented: Supports classes, objects, inheritance, polymorphism, encapsulation, abstraction.
- (ii) Rich library: Provides standard template library (STL) for DSA.

(iii) Memory management: Supports dynamic memory allocation using new and delete.

(iv) Low-level manipulation: Allow pointer operations and direct hardware interaction.

(v) Portability: Programs can run across different platforms with minimal change.

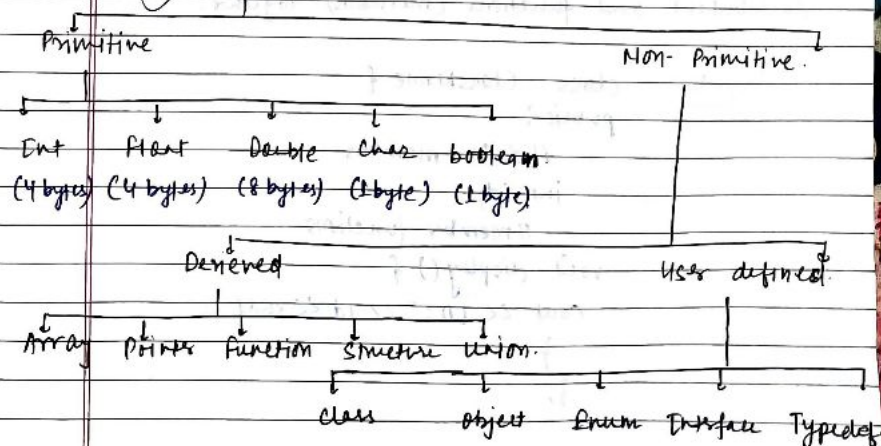
(vi) Reusability: Code can be reused through inheritance and templates.

(vii) Efficiency: Combines the speed of C with modern programming constructs.

Standard I/O objects

- * cin: Standard input stream (keyboard)
- * cout: Standard output stream (screen)
- * cerr: Standard error stream (unbuffered) use to display error messages.
- * clog: Standard error stream (buffered). used to display log/ debug messages

Data Types in C++



Variables:- A variable is a name ^{storage location} in memory that holds a value.

Static variables:- Declared using the keyword static.

* Retain their value b/w function call.

* Initialized only once.

Example:-

```
#include <iostream>
using namespace std;
void counter() {
    static int count = 0; // static variable
    count++;
    cout << "Count: " << count << endl;
}
int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

Output:-

Count: 1
Count: 2
Count: 3

Classes in C++:-

A class is a user defined data type that serves as a blueprint for creating objects. It groups data (attributes) and functions (methods) together.

Syntax:-

```
class ClassName {
public:
    // data members
    int id;
    // member functions
    void display() {
        cout << "ID: " << id << endl;
    }
};
```

Objects in C++:-

An object is an instance of a class. It represents a real world entity with state (data) & behavior (functions).

Example:-

```
#include <iostream>
using namespace std;
class student {
public:
    int rollNo;
    string name;
    void display() {
        cout << "Roll No: " << rollNo << ", Name: " << name << endl;
    }
};
int main() {
    student st; // object creation
    st.rollNo = 101;
    st.name = "Abhishek";
    st.display();
    return 0;
}
```

Output:- Roll No: 101, Name: Abhishek.

Access specifiers in C++:-

Access specifiers in C++ define the scope and visibility of class members (data and functions). They control how and where the members of a class can be accessed.

C++ provides three main access specifiers:-

- (i) Public
- (ii) Private
- (iii) Protected

(i) Public: Members declared as public are accessible from anywhere in the program.

(ii) Private: Members declared as private are accessible only within the class itself.

- * They can't be accessed directly outside the class.
- * Used for data hiding and encapsulation.

(iii) Protected: Members declared as protected are accessible within the class and by derived class (when inheritance is used).

* Since you asked without inheritance, protected behaves like private (not accessible outside the class).

Example: #include <iostream>

using namespace std;

class Demo {

public:

int publicVar; // Accessible everywhere

private:

int privateVar; // Accessible only inside class

protected:

int protectedVar; // Acts like private (not accessible outside the class)

public:

// constructor

Demo() {

publicVar = 10;

privateVar = 20;

protectedVar = 30;

}

void showData() {

cout << "Public:" << publicVar << endl;

cout << "Private:" << privateVar << endl;

cout << "Protected:" << protectedVar << endl;

}

};

int main() {

Demo obj; // Accessing public member directly.

cout << "Public(direct):" << obj.publicVar << endl;

// Accessing private / protected (Not allowed)

// cout << obj.privateVar; // ERROR

// cout << obj.protectedVar; // ERROR

// Accessing private / protected via public function (allowed)

obj.showData();

return 0;

}

Output: Public(direct): 10

Public: 10

Private: 20

Protected: 30

Constructor in C++

It is a special member function of a class that is automatically called when an object is created. Its main purpose is to initialize data members of the class.

Features:

(i) Name is same as the class name.

(ii) No return type (not even void)

(iii) Called automatically at object orientation.

(iv) Can be overloaded

(v) Usually declared in the public section.

(vi) If no constructor is defined, compiler provides a default constructor.

Syntax: class ClassName {
public:

classmate
Date _____
Page _____

```
    className() {  
        // constructor body.  
    }  
};
```

Example: #include <iostream>
using namespace std;
class Student {
public:
 int Roll;
 Student() {
 Roll = 10;
 }
 int main() {
 Student s;
 cout << s.Roll;
 return 0;
 }
};

Output:
10

Types of constructor:

(i) Default constructor: No parameters.

Ex: class A {
public:
 A() {
 cout << "Default constructor";
 }
};

(ii) Parameterized constructor: Take arguments.
class A {
public:

```
    int x;  
    A(int a) {  
        x = a;  
    }  
};
```

(iii) Copy Constructor: Initializes object using object of same class.

classmate
Date _____
Page _____

Ex: class A {
public:
 int x;
 A(int a) {
 x = a;
 }
 A(A &obj) {
 x = obj.x;
 }
};

Constructor overloading.

Ex: class A {
public:
 A() {
 cout << "Default";
 }
 A(int x) {
 cout << x;
 }
};

Dynamic Constructor: It is a constructor that allocates memory dynamically using new during object creation.

* It is mainly used when data members are pointers and memory size is decided at runtime.

* Uses new operator.

* Memory allocated inside constructor.

* Helps avoid garbage value.

* Memory released during destructor.

Example:- #include <iostream>
using namespace std;

classmate
Date _____
Page _____

```
class student {
private:
    string name;
    int ID;
public:
    student() {
        name = "Unknown";
        ID = 0;
    }
    student (string n, int x) {
        name = n;
        ID = x;
    }
    student (const student &x) {
        name = x.name;
        ID = x.ID;
    }
    void display () {
        cout << "Name=" << name << endl;
        cout << "ID=" << ID;
    }
};
```

Output:-

Name= Unknown

ID= 0

Name= Abhi

ID= 101

Name= Abhi

ID= 101

```
int main () {
    student s1;
    s1.display ();
    student s2 ("Abhi", 101);
    s2.display ();
    student s3 (s2);
    s3.display ();
    return 0;
}
```

Example:- (Dynamic Constructor)

classmate
Date _____
Page _____

```
#include <iostream>
using namespace std;
class student {
private:
    string name;
    int * marks;
    int subjects;
public:
    student (int n) {
        subjects = n;
        marks = new int [subjects];
        cout << "Memory alloc";
    }
    void set marks () {
        for (int i=0; i< subjects; i++) {
            cout << "enter marks " << i+1 << " subject";
            cin >> marks [i];
        }
    }
    ~student () {
        delete [] marks;
        cout << "Memory released";
    }
};
```

Destructor.

```
int main () {
    student s1 ("Abhi", 5);
    s1.set marks ();
    s1.display ();
    return 0;
}
```

* #include <iostream>
using namespace std;

```
class student {
private:
    string name;
    int ID;
public:
    student (string n, int x) {
        name = n;
        ID = x;
    }
    void display() {
        cout << "Name=" << name << endl << "ID=" << ID;
    }
};

int main() {
    student st("Ravi", 101);
    st.display();
    return 0;
}
```

* // Count the no. of objects created using static variable and function.

```
#include <iostream>
using namespace std;
class MyClass {
private:
    static int objCount;
public:
    MyClass() {
        objCount++;
    }
    static int getCount() {
        return objCount;
    }
};
```

```
int MyClass::objCount = 0;
int main() {
    MyClass obj1;
    MyClass obj2;
    MyClass obj3;
    cout << "Number of objects created"
    << MyClass::getCount();
    return 0;
}
```

* Code to find Area of the circle.

```
#include <iostream>
using namespace std;
class Circle {
private:
    static const double PI;
    double radius;
public:
    Circle (double r) {
        radius = r;
    }
    double area() {
        return PI * radius * radius;
    }
};

const double Circle::PI = 3.14159;
int main() {
    Circle c(5);
    cout << "Area of circle: " << c.area() << endl;
    return 0;
}
```

Ques:- Construct a class car with data members Brand and speed. Make member functions accelerate and brake. Accelerate should increase the speed by 10. Brake should decrease the speed by 5. and speed should not be negative. Initialize speed, with zero.

Objective create object of class car.
call accelerate thrice.
call brake.
display brand and speed.

include <iostream>
using namespace std;

class Car {
private:

string brand;
int speed;

public:

Car() {

cout << "Enter brand=";

cin >> brand;

speed = 0;

}

void accelerate() {

speed = speed + 10;

}

void brake() {

if (speed >= 5)

speed = speed - 5;

else

speed = 0;

}

void display() {

cout << "Brand = " << brand << " speed = " << speed;

}

int main() {

Car c1;

c1.accelerate();

c1.accelerate();

c1.accelerate();

c1.brake();

c1.display();

return 0;

}

classmate

Date

Page

*

Code for recording name, subject count and marks of a student. by dynamic constructor

include <iostream>

using namespace std;

class student {

private:

string name;

int sub-count;

int * marks;

public:

student (string n, int x)

{

name = n;

sub-count = x;

marks = new int[sub-count]; // new + Allocates Memory,

cout << "Memory allocated";

}

void setMarks() {

for (int i=0; i < sub-count; i++)

{

cout << "Enter marks";

cin >> marks[i];

}

}

void display ()

{

cout << "Marks =";

for (int i=0; i < sub-count; i++)

cout << marks[i] << " ";

}

~ student () // destructor (free memory)

{

delete [] marks;

cout << "Memory released";

}

};

classmate
Date _____
Page _____

```

int main ()
{
    student s1 ("Ajay", 5);

    s1. setMarks ();
    s1. display ();
    return 0;
}

```

Conversion → Use to change input into user defined data type
Constructor

```

#include <iostream>
using namespace std;
class student {
private:
    std string name;
    int age;
public:
    student (string n)
    {
        name = n;
        age = 18; // default value
    }
    void display ()
    {
        cout << "Name=" << name;
        cout << endl << "Age=" << age;
    }
};

```

```

int main () {
    student s1 ("Ravi"); // Explicit call
    student s2 ("Harsh"); // Implicit call.
    s1. display ();
    s2. display ();
    return 0;
}

```

(Constructor is automatically called)

Constructor overloading.

classmate
Date _____
Page _____

```

#include <iostream>
using namespace std;
class student {
private:
    string name;
    int ID;
    string subject;
public:
    student ()
    {
        cout << "Enter name";
        cin >> name;
        cout << "Enter ID";
        cin >> ID;
        cout << "Enter subject";
        cin >> subject;
    }
    student (string n)
    {
        name = n;
        cout << "Enter ID";
        cin >> ID;
        cout << "Enter subject";
        cin >> subject;
    }
    student (string n, int x).
    {
        name = n;
        ID = x;
        cout << "Enter subject";
        cin >> subject;
    }
}

```

```

void display ()
{
    cout << name;
    cout << ID;
    cout << subject;
}

int main () {
    student s1;
    student s2 ("Ajay");
    student s3 ("Ravi", 101);
    s1. display ();
    s2. display ();
    s3. display ();
    return 0;
}

```

Smart Pointers

needs file <memory>

Types: (i) unique $\rightarrow$ make_unique <data_type>
(ii) shared $\rightarrow$ make_shared <data_type>
(iii) weak $\rightarrow$

• lock() used to convert weak pointer to shared pointer.

```
#include <iostream>
#include <memory>
#include <string>
using namespace std;
class student
{
private:
    string name;
    int age;
public:
    student (string n, int a) {
        name = n;
        age = a;
    }
    void display() {
        cout << "object created";
        if (auto wptr2 = wptr1.lock()) {
            wptr2->display();
        }
        cout << name << age << endl;
    }
    ~student() {
        cout << "object destroyed";
    }
};

int main()
{
    student * rwptr1 = new student("Ravi", 19);
    student * wptr2 = rwptr1;
```

Operator overloading + to add objects.

```
#include <iostream>
using namespace std;
class Box {
    int length;
public:
    Box (int l=0) {
        length = l;
    }
    Box operator + (const Box& other) {
        return Box (length + other.length);
    }
    void display() {
        cout << "length = " << length;
    }
};

int main() {
    Box b1 (10);
    Box b2 (15);
    Box b3 = b1 + b2;
    b3.display();
    return 0;
}
```

```
#include <iostream>
using namespace std;
class Box {
    int length;
public:
    Box (int l=0) {
        length = l;
    }
    bool operator == (const Box& other) {
        return length == other.length;
    }
    void display() {
        cout << "length = " << length;
    }
};

int main() {
    Box b1 (10);
    Box b2 (15);
    cout << (b1 == b2 ? "True" : "False");
    return 0;
}
```

Ques: make a program to implement overloading, Make a class Calculator with data members operand one and operand two. make method multiply if two integers are passed do multiplication; if one integer and one string is passed, repeat the string that many times. (Ex: multiply ("a", 5) → aaaaa).

```
#include <string.h>
using namespace std;
class Calculator {
    int op1;
    int op2;
public:
    int multiply (int op1, int op2) {
        return op1 * op2;
    }
    string multiply (string s, int x) {
        string s_new = "";
        for (int i = 0; i < x; i++)
            s_new += s;
        return s_new;
    }
};

int main () {
    Calculator c1;
    cout << c1.multiply (5, 4);
    cout << c1.multiply ("hi", 5);
    return 0;
}
```

Ques: Next program Constructor overloading.
make multiple parametrized constructor & print results accordingly?

#include <iostream>
using namespace std;
class Fan {
 string color = "Brown";
 string Brand = "Khetan";
public:

void display ()

{
 cout << color << Brand;

};

int main () {

Fan f1;

f1.display ();

return 0;

}

void change_values (string c,
 string b)

{

color = c;

Brand = b;

}

~~void display ()~~

f1.change_values ("Red", "Polar");

f1.display ();

return 0;

}

* Constructor → To initialize the data members of the class.

#include <iostream>
using namespace std;

class Rectangle {

int length;

int breadth;

public:

Rectangle ()

{

length = 1; breadth = 1;

Rectangle (int l, int b)

{

length = l; breadth = b;

}

void display()

```
{  
    cout << "length" << length << endl;  
    cout << "breadth" << breadth;  
    }  
}  
int main() {  
    Rectangle r1 (5, 4);  
    r1.display();  
    return 0;  
}
```

classmate

Date _____
Page _____

Single Inheritance

classmate

Date _____
Page _____

```
#include <iostream>  
using namespace std;  
class parent  
{  
    public:  
        parent()  
        {  
            cout << "constructor parent" << endl;  
        }  
};  
class child: public parent  
{  
    public:  
        child()  
        {  
            cout << "constructor child" << endl;  
        }  
};  
int main()  
{  
    child c;  
    return 0;  
}
```

```
* #include <iostream>  
using namespace std;  
class Parent {  
    protected: string name; int age = 0;  
    public:  
        parent (string n = "", int age = 0)  
        {  
            name = n; age = a;  
        }  
}
```

void display() {

cout << "name=" << name << ", age=" << age << endl;

};

class child : public parent {

string name; int age;

public:

child() {

cout << "child constructor";

cout << "Enter name";

cin >> name;

cout << "Enter age";

cin >> age;

}

void show() {

cout << "name=" << name << ", age=" << age << endl;

}

};

int main() {

parent p("ravi", 45);

p.display();

child c;

c.show();

return 0;

};

Pass values b/w constructors in inheritance

#include <iostream>

using namespace std;

class father {

protected:

string f_name;

int f_age;

public:

father(string n, int a)

{

f_name = n;

f_age = a;

}

};

class mother {

protected:

string m_name;

int m_age;

public:

mother(string n, int a)

{

m_name = n;

m_age = a;

}

};

class child : public father

{

string c_name;

int c_age;

public:

child(string n, string fn, string mn, int a, int fa, int ma) : father(fn, fa), mother(mn, ma)

{

c_name = n;

c_age = a;

void display()

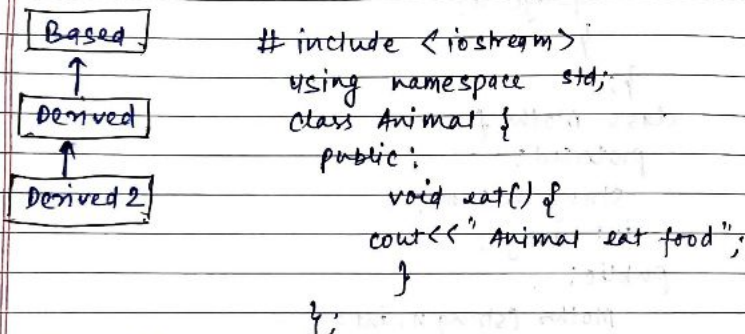
```

{
    cout << " ";
}
};

int main() {
    child c("Baby", "Ravi", "Terna", 10, 28, 22);
    c.display();
    return 0;
}

```

Multilevel Inheritance



```

class Mammal: public Animal
{
public:
    void breathe() {
        cout << "mammal breathes air";
    }
};

```

```

class Dog: public Mammal {
public:
    void speak() {
        cout << "Dog barks";
    }
};

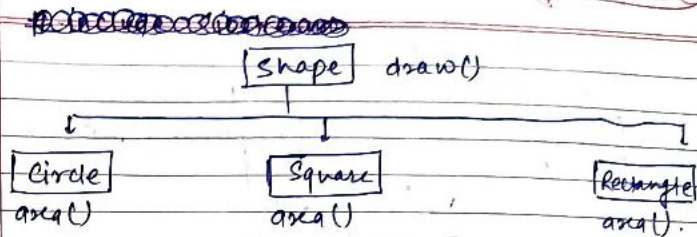
```

```

int main()
{
    Dog d;
    d.eat();
    d.breathe();
    d.speak();
    return 0;
}

```

Hierarchical Inheritance



```

int main()
{
    Circle c; Square s; Rectangle r;
    c.draw(); c.oxa(); s.draw(); s.oxa(); r.draw(); r.oxa();
    return 0;
}

```

```

#include <iostream>
using namespace std;
class shape {
public:
    void draw() {
        cout << "I am drawing";
    }
};
class Circle: public shape {
public:
    void oxa() {
        cout << "x r 2";
    }
};

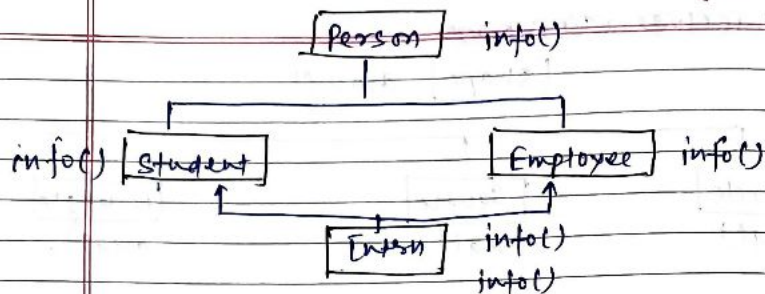
```

```

int main()
{
    Circle c;
    c.draw();
    c.oxa();
    return 0;
}

```

Hybrid Inheritance



```

#include <iostream>
using namespace std;
class Person {
public:
    void info() {
        cout << "I am Person";
    }
};
class Student: virtual public Person
{
public:
    void story() {
        cout << "I am studying";
    }
};

```

```

class Employee: virtual public Person
{
public:
    void work() {
        cout << "I am working";
    }
};
class Intern: public Student, public Employee
{
public:
    void learn() {

```

```

cout << "I am learning";
}
};

```

```

int main()
{
    Intern obj;
    obj.info();
    obj.study();
    obj.work();
    obj.learn();
    return 0;
}

```

Inline function

lambda fcn in Python.

```

inline int square(int x) {
    return x*x;
}
int main()
{
    int num;
    cout << "Enter number";
    cin >> num;
    cout << "square of num" << square(num);
    return 0;
}

```

Friend fcn is not a member of class but it can access private, protected, public members of a class

```

#include <iostream>
using namespace std;
class Box {
    int length;
public:
    Box(int l)
    {
        length = l;
    }
}

```

friend void printlen (Box b);

};

void printlen (Box b)

```
{
    cout << "length = " << b.length;
}
```

int main()

{

Box b(10);

printlen(b);

return 0;

}

*

#include <iostream>

using namespace std;

class Base {

public:

virtual void info()

{

cout << "Base Info";

}

};

class Derive: public Base

{

public:

void info() override {

cout << "Derive info";

}

};

int main()

{

Base *ptr;

Derive d;

ptr = &d;

ptr->info(); // Derive info.

return 0;

}

classmate

Date
Page

Virtual Function

classmate

Date
Page

It implements run time polymorphism. We declare a function using virtual keyword in base class then we override that function in derive class. We use base class pointer to hold address of derive object.

Pure virtual function

~~Syntax~~ Syntax: virtual void fun_name(parameters)=0;

Abstract class

#include <iostream>

using namespace std;

class Shape {

public:

void draw() {

cout << "I am drawing";

}

virtual void area()=0;

};

class Circle: public Shape

{

public:

void area() override {

cout << "circle area is 3.14";

}

};

int main()

{

Shape *s;

Circle c;

s = &c;

s->draw(); // I am drawing.

s->area(); // circle area is 3.14.

return 0; }

include <iostream>
using namespace std;

class shape {

public:

void draw() {

cout << "I am drawing";

}

virtual void area()=0;

};

class Circle: public shape {

int radius;

public:

Circle(int r): radius(r) {}

void area() override.

{ cout << "Area=" << 3.14 * radius * radius;

}

};

int main() {

shape *s;

Circle c(5);

s = &c;

s->draw(); // I am drawing.

s->area(); // Area= 75.42

return 0;

}

classmate

Date

Page

Access specifier

classmate

Date

Page

include <iostream>

using namespace std;

class Base {

private:

int password = 788;

protected

int money = 100000;

public:

String scooter = "Activa";

}

class Derive: private Base {

public:

void display()

{

cout << money << scooter;

}

};

class

#include <iostream>

using namespace std;

class Grand-father {

public:

string name;

Grand-father (string n) {

name = n;

}

};

class Father: public Grand-father {

public:

string fname;

Father (string n, string fn): Grand-father (fn) {

fname = n;

}

};

class Child: public Father {

public:

string cname;

Child (string n, string fn, string gfn): Father (fn, gfn) {

cname = n;

}

void display() {

cout << "Child name=" << cname << endl;

cout << "Father name=" << fname << endl;

cout << "Grandfather name=" << gname << endl;

}

};

int main() {

Child c ("Chotu", "Madhav", "Hemant");

c.display();

return 0;

}

Hierarchical Inheritance

#include <iostream>

using namespace std;

class Shape {

public:

Shape() {

cout << "Shape Constructor" << endl;

}

void draw() {

cout << "I am drawing" << endl;

}

};

class Circle: public Shape {

int radius;

public:

Circle (int r) {

radius = r;

draw();

}

void area() {

cout << "Area circle=" << 3.14 * radius * radius << endl;

};

class Rectangle: public Shape {

int length;

int breadth;

public:

Rectangle (int len, int bre) {

length = len;

breadth = bre;

draw();

}

void area() {

cout << "Area rectangle=" << length * breadth << endl;

}

};

```
int main() {
    Circle c(5); Rectangle r(5,7);
    c.area();
    r.area();
    return 0;
}
```

11 friend function in another class make that class friend

```
#include <iostream>
using namespace std;
class B; // forward declaration.
class A {
private:
    int password = 786;
public:
    A() {
        cout << "A constructor" << endl;
    }
    friend class B;
};
class B {
public:
    void reveal_password(A obj) {
        cout << "Password = " << obj.password << endl;
    }
};
int main() {
    A obj-a;
    B obj-b;
    obj-b.reveal_password(obj-a);
    return 0;
}
```

State Diagram.

classmate
Date _____
Page _____

Definition: It is a graph whose nodes represent unique states and whose directed arcs represent transition from one state to another based on an event.

Components of state diagram:-

(i) State 

↳ It is a recognizable state of an object at a particular time.

(ii) Transition event 

(iii) Change event when(event)

(iv) Time event at time (event)

(v) Initial state point ●

(vi) Final state point ○

(vii) Guarded Transition [boolean expression]

Function template

classmate

Date _____
Page _____

```
#include <iostream>
using namespace std;
template <typename T>
T mymax (T x, T y) {
    if (x > y)
        return x;
    else
        return y;
}
// or
return (x > y) ? x : y; // Ternary operator
}
```

```
int main() {
    cout << "Integer max=" << mymax<int>(5, 5.3)
        << endl;
    cout << "Double max=" << mymax<double>(8.4, 8.2)
        << endl;
    cout << "Char max=" << mymax<char>('h', 'm')
        << endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;
template <typename T>
T findmax (T arr[], int s) {
    T var = arr[0];
    for (int i = 1; i < s; i++) {
        if (arr[i] > var) {
            var = arr[i];
        }
    }
    return var;
}
```

int main () {

```
int arr[5] = {3, 5, 7, 8, 1};
double arrd[] = {4.4, 2.2, 8.8, 5.5};
char arrc[] = {'a', 'e', 'a', 'r'};
```

```
cout << "Integer arr max=" << findmax<int>(arr, 5)
        << endl;
```

```
cout << "Double arr max=" << findmax<double>(arrd, 4) << endl;
cout << "Char arr max=" << findmax<char>(arrc, 4);
return 0;
}
```

Class Template

```
#include <iostream>
using namespace std;
template <typename T>
class Array {
private:
    T* ptr;
    int size;
```

public:

```
Array (T arr[], int s) {
    ptr = new T[s];
    size = s;
    for (int i = 0; i < size; i++) {
        ptr[i] = arr[i]; // I am using pointer as array.
    }
}
```

```
void print() {
    for (int i = 0; i < size; i++)
        cout << ptr[i]; // *(ptr + i)
    cout << endl;
}
```

```
~Array() { // destructor
    delete[] ptr;
}
```

```
int main () {
```

```
    int arr[] = {4, 5, 6, 7, 8};
```

```
    char arrc[] = {'h', 'e', 'l', 'l', 'o'};
```

```
    Array<int> a(arr, 5);
```

```
    Array<char> b(arrc, 5);
```

```
    a.print();
```

```
    b.print();
```

```
    return 0;
```

```
}
```

*

```
#include <iostream>
```

```
#include <typeinfo>
```

```
using namespace std;
```

```
template <typename T, typename U>
```

```
class A {
```

```
    T x;
```

```
    U y;
```

```
public:
```

```
    A() {
```

```
        cout << "Constructor called" << endl;
```

```
        cout << "Data type x = " << typeid(x).name() << endl;
```

```
        cout << "Data type y = " << typeid(y).name() << endl;
```

```
    };
```

```
int main() {
```

```
    A<int, char> a;
```

```
    A<char, double> b;
```

```
    return 0;
```

```
}
```

classmate

Date
Page

```
#include <iostream>
```

```
using namespace std;
```

```
template <typename T1, typename T2>
```

```
class Pair {
```

```
    T1 x; T2 y;
```

```
public:
```

```
    Pair(T1 a, T2 b) : x(a), y(b) {}
```

```
    void display() { cout << "x = " << x << "y = " << y << endl; }
```

```
    void printsum() {
```

```
        cout << "Sum = " << x + y << endl; }
```

```
};
```

```
int main() {
```

```
    Pair<int, double> a(5, 3.8);
```

```
    a.display();
```

```
    a.printsum();
```

```
    Pair<string, string> b("work", "hard");
```

```
    b.display();
```

```
    b.printsum();
```

```
    return 0;
```

```
}
```

```
#include <iostream>
```

```
using namespace std;
```

```
template <typename T1, typename T2>
```

```
auto multiply(T1 x, T2 y) -> decltype decltype(x * y) {
```

```
    return x * y;
```

```
}
```

```
int main() {
```

```
    cout << "Ans: " << multiply<int, double>(4, 3.5) << endl;
```

```
    cout << "Ans: " << multiply<float, float>(4.2f, 3.5f) << endl;
```

```
    return 0;
```

```
}
```

classmate

Date
Page

```
#include <iostream>
using namespace std;
template <typename T>
```

Exception Handling:-

```
#include <iostream>
using namespace std;
int main () {
    try {
        int age;
        cout << "enter age";
        cin >> age;
        if (age < 18) {
            throw age;
        }
        cout << "Eligible to vote";
    }
    catch (int e) {
        cout << "Not Eligible, Age is: " << e;
    }
    return 0;
}
```

Zero Division Error:-

// invalid_argument standard error.

```
#include <stdexcept>
#include <iostream>
using namespace std;
double divide (double nume, double deno) {
    if (deno == 0) {
        throw invalid_argument ("Divide zero not allowed");
    }
    return nume / deno;
}
```

```
int main () {
```

```
double nume, deno, res;
cout << "enter numerator: " << endl; cin >> nume;
cout << "enter denominator: " << endl; cin >> deno;
try {
    res = divide (nume, deno);
    cout << "Result = " << res << endl;
}
catch (const invalid_argument & e) {
    cerr << "Error" << e.what() << endl;
}
catch (...) {
    cerr << "Unknown error" << endl;
}
return 0;
}
```

```
#include <stdexcept>
```

```
#include <iostream>
using namespace std;
int get_element (int arr[], int size, int index) {
    if (index < 0 || index >= size) {
        throw out_of_range ("index value out of range");
    }
    return arr[index];
}
```

```
int main () {
```

```
int arr[5] = {9, 8, 7, 6, 5};
int index;
cout << "enter index: ";
cin >> index;
try {
    int val = get_element (arr, 5, index);
    cout << "element = " << val;
}
}
```

```

    cerr << "error!" << e.what() << endl;
    catch (...){
        cerr << "unknown error" << endl;
    }
    return 0;
}

```

Custom Exceptions

Programmer make their own error handling exception class.

The header file used is <exception>.

It is recommended to inherit exception class.

```

#include <exception>
#include <iostream>
using namespace std;
class NegativeVE : public exception{
    int value;
public:
    NegativeVE(int x): value(x){
        const char* what() const noexcept override{
            return "Bhai error aa gaya, negative value";
        }
        int getValue() const{
            return value;
        }
    };
    void checkValue(int x){
        if (x < 0){
            throw NegativeVE(x);
        }
        else
            cout << "value is" << x << endl;
    }
}

```

classmate
 Date _____
 Page _____

```

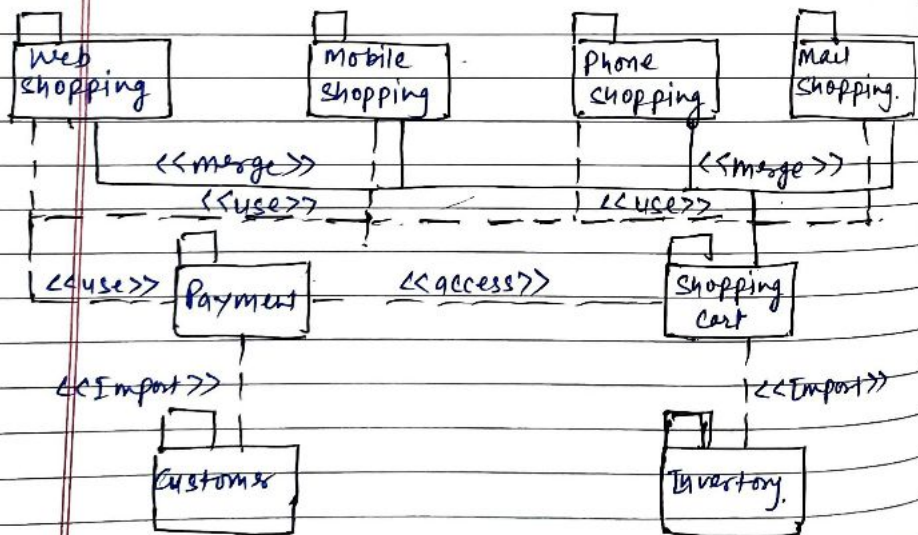
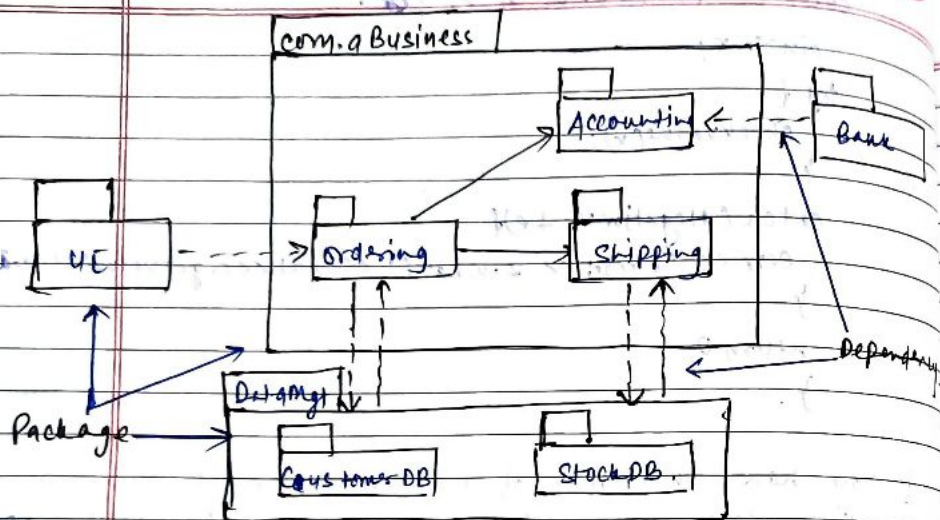
int main(){
    int x;
    cout << "Enter value of x:";
    cin >> x;
    try{
        checkValue(x);
    }
    catch (NegativeVE &e){
        cerr << "Error" << e.what() << endl << e.getValue() << endl;
    }
    return 0;
}

```

Q. You have an array of marks, implement an error handling class, check all marks & ensure to raise error if marks greater than 100 or less than zero?

Package Diagram

classmate
Date _____
Page _____



Unit - IV

Diagrams
Package
Component
Deployment

Unit V classmate
Date _____
Page _____
STL (Standard Template Library)
Has 3 components

- (i) Containers
- (ii) Algorithms
- (iii) Iterators.

Containers:

Types of Containers:

- (i) Sequence Containers: Vector, List, forward list, Array.
- (ii) Containers Adaptors: Stack, Queue, Priority Queue
- (iii) Associative Containers: Set, multiset, map, multimap.
- (iv) Unordered Associative Containers: unordered set, unordered multiset, unordered map, unordered multimap.

```

#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v; // empty vector
    vector<int> v1 = {4, 5, 6};
    vector<int> v2 {5, 10};
    for (int x: v) cout << x << " ";
    cout << endl;
    for (int x: v1) cout << x << " ";
    cout << endl;
    for (int x: v2) cout << x << " ";
    return 0;
}
    
```

Output: value at v[2] index 2

```
#include <vector>
#include <iostream>
using namespace std;
int main() {
    vector<int> v;
    for (int i=0; i<4; i++) {
        cout << "enter element: ";
        cin >> x; v.push_back(x);
    }
    for (int x: v) cout << endl;
    cout << x << " ";
    return 0;
}

for (int i=0; i<int(v.size()); i++) {
    for (int j=i+1; j<int(v.size()); j++) {
        if (v[i] == v[j]) v.erase(v.begin() + j);
        else j++;
    }
}

for (int x: v)
    cout << x << " ";
```

```
#include <list>
#include <iostream>
using namespace std;
int main() {
    list<int> mylist = {2, 4, 5, 6};
    cout << mylist.front() << endl;
    cout << mylist.back() << endl;
    cout << "size = " << mylist.size() << endl;
    mylist.push_front(2);
    mylist.push_back(7);
    for (int x: mylist) cout << x << " "; cout << endl;
    mylist.pop_front();
    mylist.pop_back();
    for (int x: mylist) cout << x << " "; cout << endl;
    return 0;
}
```

```
#include <list>
#include <iostream>
using namespace std;
int main() {
    list<int> mylist = {10, 20, 30, 40, 50};
    for (int x: mylist)
        cout << x << " ";
    cout << endl;
    auto it = mylist.begin();
    advance(it, 1);
    mylist.insert(it, 15); cout << "after insert" << endl;
    advance(it, 4);
    mylist.insert(it, 45);
    for (int x: mylist) cout << x << " "; cout << endl;
    return 0;
}

mylist.remove_if(isOdd);
cout << "After removing odd values" << endl;
for (int x: mylist) cout << x << " ";
cout << endl;

auto it = mylist.begin();
advance(it, 2);
mylist.erase(it);
cout << "after deleting" << endl;
```

```
#include <list>
#include <iostream>
using namespace std;
bool isVowel(char x) {
    if (x == 'a' || x == 'e' || x == 'i' || x == 'o' || x == 'u')
        return true;
    else return false;
}

int main() {
    list<char> myword = {'h', 'a', 'p', 'p', 'y', 'h', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd'};
    for (char x: myword) cout << x << " "; cout << endl;
    myword.remove_if(isVowel);
    for (char x: myword) cout << x << " "; cout << endl;
    return 0;
}
```

Inheritance

Inheritance is a mechanism in OOP where one class (child / derived class) acquires properties and behaviours (fields and methods) of another class (parent / base class).

Purpose of Inheritance:

- * Promotes code reusability.
- * Establish an IS A relationship b/w classes
- * Supports Polymorphism and extensibility.

Types of Inheritance

(i) Single Inheritance: In this a child class inherits from only one parent class

- * Use Case: Simple hierarchical relationships.

Example:

- * Simple, avoid ambiguity.

```
#include <iostream>
using namespace std;
class Animal {
public:
    void speak() {
        cout << "Animal makes a sound" << endl;
    }
};
class Dog: public Animal {
public:
    void speak() {
        cout << "Dog barks" << endl;
    }
};
```

```
int main() {
    Dog d;
    d.speak();
    return 0;
}
```

(ii) Multiple Inheritance: In this a child class inherits from more than one parent class.

- * Use Case: When a class needs features from multiple sources.

- * Powerful but can cause conflicts.

Example:

```
#include <iostream>
using namespace std;
class Father {
public:
    void gardening() {
        cout << "Father: Gardening" << endl;
    }
};
class Mother {
public:
    void cooking() {
        cout << "Mother: Cooking" << endl;
    }
};
class Child: public Father, public Mother {
public:
    void painting() {
        cout << "Child: Painting" << endl;
    }
};
int main() {
    Child c;
    c.gardening(); // Output: Father: Gardening
    c.cooking();   // Output: Mother: Cooking
    c.painting();  // Output: Child: Painting
    return 0;
}
```

(iii) Multilevel Inheritance:-

Multilevel Inheritance is a concept in OOP where a class is derived from another class, and then another class is derived from that derived class forming a chain.

Class A $\longrightarrow$ class B $\longrightarrow$ class C
(Base) (Derived) (Derived)

Example:-

```
#include <iostream>
using namespace std;
class Animal {
public:
    void eat() {
        cout << "Animal eats food" << endl;
    }
};
class Mammal: public Animal {
public:
    void breathe() {
        cout << "Mammal breathes air" << endl;
    }
};
class Dog: public Mammal {
public:
    void bark() {
        cout << "Dog barks" << endl;
    }
};
int main() {
    Dog d;
    d.eat();
    d.breathe();
    d.bark();
    return 0;
}
```

(iv) Hierarchical Inheritance-

In this multiple classes inherits from a single base class

* Use Case: When different derived classes share common features of a base class

Example:-

```
#include <iostream>
using namespace std;
class Shape {
public:
    void draw() {
        cout << "Drawing a shape" << endl;
    }
};
class Circle: public Shape {
public:
    void area() {
        cout << "Area of circle =  $\pi r^2$ " << endl;
    }
};
class Rectangle: public Shape {
public:
    void area() {
        cout << "Area of Rectangle =  $l \times b$ " << endl;
    }
};
int main() {
    Circle c;
    Rectangle r;
    c.draw();
    c.area();
    r.draw();
    r.area();
    return 0;
}
```

(M) Hybrid Inheritance:

* A combination of two or more than two types of inheritance is called hybrid Inheritance (e.g. multiteret + multiple).

* Use Case: Complex systems where classes need to inherit from multiple hierarchies.

* It can lead to ambiguity (diamond problem) solved using Virtual Inheritance.

Example: #include <iostream>

using namespace std;

class Person {

public:

void info() {

cout << "I am a person" << endl;

}};

class Student: virtual public Person {

public:

void study() {

cout << "I am studying" << endl;

}};

class Employee: virtual public Person {

public:

void work() {

cout << "I am working" << endl;

}};

class Intern: public Student, public Employee {

public:

void note() {

cout << "I am an intern" << endl;

}};

int main() {

Intern i;

i.info(); // From Person (via virtual inheritance, avoids ambiguity)

i.study(); // From Student

i.work(); // From Employee

i.note(); // From Intern.

return 0;

}

Diamond Problem: Occurs when multiple paths lead to the same base class, solved using virtual keyword in C++.

Inline Functions:

Functions defined with the inline keyword request the compiler to insert the function's code directly at the call site, reducing function call overhead.

Merits: * Faster execution (avoids function call overhead)

* Useful for small, frequently used functions.

Demerits: * Not suitable for large functions

* Compiler may ignore inline request

Example: #include <iostream>

using namespace std;

inline int square(int x) {

return x*x;

}

int main() {

cout << "Square of 5 = " << square(5) << endl;

return 0;

}

Stack Containers

```
#include <stack>
#include <iostream>
using namespace std;
int main() {
    stack<int> st;
    st.push(10);
    st.push(20);
    st.push(30);
    cout << st.top() << endl;
    st.pop();
    cout << st.top();
    return 0;
}
```

```
#include <stack>
#include <iostream>
using namespace std;
int main() {
    int n, x;
    cout << "Enter how many elements: ";
    cin >> n;
    stack<int> st;
    for (int i=0; i<n; i++) {
        cout << "Enter element: ";
        cin >> x;
        st.push(x);
    }
    while (!st.empty()) {
        cout << st.top();
        cout << endl;
        st.pop();
    }
    return 0;
}
```

// factorial using recursion.

```
#include <stack>
#include <iostream>
using namespace std;
int fact(int n, stack<int> & callstack) {
    callstack callstack.push(n);
    stack<int> temp = callstack;
    cout << "callstack = ";
    while (!temp.empty()) {
        cout << temp.pop() << endl;
        temp.pop();
    }
    if (n==0 || n==1) {
        callstack.pop();
        return 1;
    }
    int result = n * fact(n-1, callstack);
    callstack.pop();
    return result;
}

int main() {
    int n;
    cout << "Enter factorial number: ";
    cin >> n;
    stack<int> callstack;
    cout << "factorial = " << fact(n, callstack);
    return 0;
}
```

Access specifiers and their Role in Inheritance

* Access Specifiers: Access specifiers in C++ define the scope and visibility of class members (data & functions).

They control how and where members of class can be accessed.

They play a crucial role in inheritance because they determine how members of a base class are accessible in a derived class and to outside class.

(1) Public Inheritance

Public Members → Remain public in derived class

Protected members → " " " " " "

Private Members → Not accessible directly in the derived class.

Use Case: Model an 'is-a' relationship (ex: Car is a vehicle).

(2) Protected Inheritance

Public Members → Become protected in the derived class

Protected Members → " " " " " "

Private members → Not accessible directly.

Use Case: Restricts outside access but allows derived classes to use base functionality internally.

(3) Private Inheritance

Public Members → Becomes private in derived class

Protected Members → " " " " " "

Private Members → Not accessible directly

Friend Functions

* A friend function is not a member of a class but has access to its private and protected members.

Use Case: When external functions need access to private data.

```
Ex:- #include <iostream>
using namespace std;
class Box {
private:
    int length;
public:
    Box(int l); length(l) {}
    friend void printLength(Box b); // Friend Function Declaration.
}
void printLength(Box b) {
    cout << "length of box = " << b.length << endl;
}
int main() {
    Box b(10);
    printLength(b);
    return 0;
}
```

Virtual Functions

* Declared with the virtual keyword in a base class, allowing derived classes to override them.

* Enables run time polymorphism, meaning the program decides which function to call while the program is running, not at compile time.

Use Case: When behaviour differs across derived class.

* Virtual functions usually work with base class pointers.

Ex:- #include <iostream>
 using namespace std;
 class Animal {
 public:
 virtual void speak() {
 cout << "Animal makes sound" << endl;
 }
 };
 class Dog: public Animal {
 public:
 void speak() override {
 cout << "Dog barks" << endl;
 }
 };
 int main() {
 Animal *a;
 Dog d;
 a = &d;
 a->speak(); // Run time Polymorphism
 return 0; // Output: Dog barks
 }

NOTE:- The reason we often use a base class pointer to store the address of a derived class object in the context of virtual functions is tied directly to runtime polymorphism.

- * A base class pointer can point to objects of the base class and any of its derived classes.
- * When the base class has a virtual function, the compiler sets up a mechanism so that calls to the function are resolved at runtime, depending on the actual type of the object being pointed to.

Ex:- #include <iostream>
 using namespace std;
 class Base {
 public:
 virtual void show() {
 cout << "Base class show()" << endl;
 }
 };
 class Derived: public Base {
 public:
 void show() override {
 cout << "Derived class show()" << endl;
 }
 };
 int main() {
 Base *ptr; // Base class pointer.
 Derived d; // Derived class object
 ptr = &d; // Pointer pts to derived object
 ptr->show(); // Calls derived::show() because of virtual function.
 return 0;
 }

Pure Virtual Functions:-

- * A virtual function with = 0 is a pure virtual function. It makes the class abstract.
- * Abstract class: A class containing at least one pure virtual function. Cannot be instantiated directly.

Use Case:- Defines an interface for derived class.

Ex:- #include <iostream> (Also example of abstract classes in C++)
 using namespace std;
 class Shape {
 public:
 virtual void area() = 0; // Pure virtual function.
 };

class Circle: Public shape {

public:

void area() override {

cout << "Area of circle = πr^2 " << endl;

};

class Rectangle: Public shape {

public:

void area() override {

cout << "Area of rectangle = $l * b$ " << endl;

};

int main() {

shape *s;

[On Exam time: see notes]

Circle c;

Table (for MCQ).

Rectangle r;

s = &c;

s->area();

s = &r;

s->area();

return 0;

}

Abstract classes in C++

- * An abstract class in C++ is a class that cannot be instantiated directly.
- * It serves as a base class for other classes and provides a framework for derived classes.
- * They are typically used to define interfaces or common behaviour.

Declaring abstract class:

In C++, a class becomes abstract if it contains at least one pure virtual function.

Abstract class v/s Interface (in C++) :-

- * C++ does not have separate interface keyword like Java or C#.
- * Interfaces are implemented using abstract classes with only pure virtual functions.

Use Case:-

Polymorphism:- Abstract classes allow writing generic code that works with different derived class.

Frame Works:- Define a common contract for multiple implementations.

Code Organization:- separate abstract class (what to do) from derived class (how to do it).

sort(v.begin(), v.end());

for (int x: v) cout << x << " "; cout << endl;

int result = bsearch(v, target);

if (result != -1)

cout << "element found at:" << result;

else

cout << "element not found";

return 0;

}

Generic Programming

It is a style of writing code where you design algorithms and data structures to work with any data type, rather than a specific one. It makes your code reusable, flexible and type safe.

Advantages of Generic Programming

- (i) Code Reusability.
- (ii) Avoid function overloading.
- (iii) Once written it can be used for multiple times and case.

* Generic programming can be implemented in C++ using Templates.

Generic Function Using Template

A generic function using templates (in C++) allows you to write a single function that works with any data type, instead of creating separate functions for each type.

template <typename T> :- Defines a generic type T.

T can be replaced with int, float, double etc.

Ex:-
#include <iostream>
using namespace std;
template <typename T>
T add (T a, T b) {
return a+b; }

int main() {
cout << add(3, 5) << endl;
cout << add(2.5, 4.1) << endl;
return 0;
}

Generic class using Template :-

A generic class using templates (in C++) allows you to create a class that can work with any data type, instead of being limited to one specific type.

Ex:- #include <iostream>
using namespace std;
template <typename T>
class Pair {
 T a, T b;

```
public: Pair(Tx, Ty) {  
    a = x; b = y;  
}  
void display() {  
    cout << "A: " << a << ", B: " << b << endl;  
} }  
int main() {  
    Pair<int> p1(10, 20);  
    p1.display();  
    Pair<float> p2(2.5, 3.7);  
    p2.display();  
    return 0;  
}
```

Ex:- example of template Array class.

```
#include <iostream>  
using namespace std;  
template <typename T>  
class Array {  
private:  
    T* ptr;  
    int size;
```

classmate
Date _____
Page _____

public: // constructor defined inside the classmate
Array (T arr[], int s) {

```
    ptr = new T[s];  
    size = s;  
    for (int i = 0; i < size; i++)  
        ptr[i] = arr[i];  
}
```

```
void print() { // Print function defined inside the class  
    for (int i = 0; i < size; i++)  
        cout << " " << ptr[i];  
    cout << endl;  
}
```

```
~Array() { // Destructor to free memory.  
    delete [] ptr;  
} }  
int arr[5] = {1, 2, 3, 4, 5};  
Array<int> a(arr, 5);  
a.print();  
return 0;  
}
```

Working with multi-type generics :-

In C++ multi-type generics are implemented using templates with ~~using~~ multiple parameters.

* This allows you to define a class or function that can work with multiple types at once.

Ex:- #include <iostream>
using namespace std;
template <class T, class U>
class A {

```
    T x; U y;  
public: A() {  
    cout << "Constructor Called" << endl;  
} };
```

```
int main() {  
    A<char, char> a;  
    A<int, int> b;
```

Output:
Constructor Called
Constructor Called.

Ex:

include <iostream>

using namespace std;

template < typename T, typename U >

void show (T x, U y) {

cout << x << " * << y << endl;

}

int main () {

show (100, "World");

show (3.14, 'A');

}

Exception handling in C++

Exception handling in C++ allows you to detect and handle runtime errors gracefully without abruptly terminating the program:-

It uses three main keywords:-

(i) try → Defines a block of code to test for errors

(ii) throw → Signals that an exception (error) has occurred.

(iii) catch → Defines a block of code to handle the thrown exception.

Syntax of Exception Handling:

try {

// code that may throw an exception.

throw exception_type (value);

}

catch (exception_type e)

// code to handle the exception.

classmate

Date
Page

Types of Exceptions:-

classmate

Date
Page

(i) Built in Exceptions: These are basic data types thrown as exceptions.

* Simple values like integers or strings can be thrown, but they lack detailed information.

Example:- (Division by zero)

include <iostream>

using namespace std;

int main () {

int a=10, b=0;

try {

if (b==0)

throw b; // built in type (int) exception.

cout << a/b;

}

catch (int e) {

cout << "Exception Caught: Division by zero, value=" << e;

}

return 0;

}

Example: (String Exception)

include <iostream>

using namespace std;

int main () {

try {

throw "Error occurred";

}

catch (const char* msg) {

cout << "Exception: " << msg;

}

return 0;

}

(ii) Custom exceptions:-

Custom exceptions are user defined exception classes

that you create to handle specific error situations in your program.

Example:- (Invalid Age Exception)

```
#include <iostream>
using namespace std;
class InvalidAge {
public:
    const const char* msg() {
        return "Age must be 18 or above";
    }
};

int main() {
    int age = 15;
    try {
        if (age < 18)
            throw InvalidAge();
        cout << "Access granted";
    }
    catch (InvalidAge e) {
        cout << e.msg();
    }
}
```

Example:- (Negative Number Exception)

```
#include <iostream>
using namespace std;
class NegativeNumber {
public:
    const char* message() {
        return "Negative numbers not allowed";
    }
}
```

```
int main() {
    int num = -5;
    try {
        if (num > 0);
        throw NegativeNumber();
        cout << num;
    }
    catch (NegativeNumber e) {
        cout << e.message();
    }
}
```

Example:- (Custom Exception using std::exception)

```
#include <iostream>
#include <exception>
using namespace std;
class MyException: public exception {
public:
    const char* what() const noexcept override {
        return "This is standard custom exception";
    }
};

int main() {
    try {
        throw MyException();
    }
    catch (exception &e) {
        cout << e.what();
    }
}
```

Standard Exceptions

Standard exceptions in C++ are predefined exception provided by the C++ standard library (mainly in `<exception>` and `<stdexcept>`).

They help handle common run-time error in a structured way.

Base Class: `std::exception`

- * All standard exceptions are derived from `std::exception`.
- * It provides a virtual function `what()` which returns the error message.

Common Standard exceptions

From `<exception>`

- (i) `std::exception` → Base class
- (ii) `std::bad_alloc` → Memory allocation failure
- (iii) `std::bad_cast` → Invalid type casting.
- (iv) `std::ios_base::failure` → Raised for input/output stream error from `<stdexcept>`

- (i) `std::runtime_error` → General runtime error
- (ii) `std::logic_error` → logical mistake in program

- (iii) `std::invalid_argument` → Invalid input

- (iv) `std::out_of_range` → Index out of bounds

- (v) `std::overflow_error` → Overflow Conditions

- (vi) `std::underflow_error` → Underflow Conditions.

Example:- (Using `std::runtime_error`)

```
#include <iostream>
#include <stdexcept>
using namespace std;
int main() {
    try {
        throw runtime_error("Runtime error occurred");
    }
    catch(exception &e) {
        cout << e.what();
    }
}
```

Example:- Using `out-of-range`

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v = {1, 2, 3};
    try {
        cout << v.at(5); // out of range
    }
    catch(out_of_range &e) {
        cout << "Exception: " << e.what();
    }
}
```

Example:- Using `bad_alloc`

```
#include <iostream>
using namespace std;
int main() {
    try {
        int *p = new int[1000000000000]; // very large allocation.
    }
    catch(bad_alloc &e) {
        cout << "Memory allocation failed";
    }
}
```

STL Containers (Standard Template Library)

- * STL (Standard Template Library) contains in C++ data structures that store collection of objects.
- * It is a collection of pre-built classes and functions that make it easy to manage data using common data structures.
- * It saves time and effort by providing ready to use, efficient algorithms and containers.

Components of STL

- (i) Containers: Containers store data (collection of objects).

Types: (a) Sequence Containers $\rightarrow$ vector, list, deque

(b) Associative Containers $\rightarrow$ set, map, multimap.

(c) Unordered Containers $\rightarrow$ unordered_set, unordered_map

(d) Container Adapters $\rightarrow$ stack, queue.

- (ii) Algorithms: Algorithms perform operations on data in containers.

Common algorithms:

Sort $\rightarrow$ Arranges elements in ascending order.

Binary search $\rightarrow$ Check whether a value exists in a sorted array.

Find $\rightarrow$ Searches for the first occurrence of given value.

Count $\rightarrow$ Counts how many times a value appeared in given range.

Count of elements in given range.

Accumulate $\rightarrow$ Computes the sum of all elements in the range.

Unique $\rightarrow$ Removes duplicate elements.

Lower bound $\rightarrow$ Return iterator to the first element $\rightarrow$ value in sorted range.

Upper bound $\rightarrow$ Return iterator to the first element $\rightarrow$ value in a sorted range.

Replace $\rightarrow$ Replaces all occurrences of old value with a new value.

- (iii) Iterators: Iterators are the pointer like objects that used to point to the memory address of STL containers.

- * They are one of the most important components that contribute the most in connecting the STL algorithms with the containers.

- * Iterators are defined inside the `<iterator>` header file.

Benefits of C++ STL:

(i) Saves time and effort.

(ii) Reliable and Tested.

(iii) Fast and Efficient.

(iv) Reusability.

(v) Built-in Algorithms.

Sequence Containers

Sequence Containers store elements in linear order, just like a sequence.

- * Each element has a unique position and insertion order is preserved.

Types of Sequence Containers

(i) vector (Dynamic Array): A vector is a dynamic array in C++ STL that can grow or shrink in size automatically. It is most commonly used sequence containers.

- * It allows random access.
- * Slows insertion/deletion in the middle.
- * Fast insertion at the end.

Important Functions

push_back() → Add element at end

pop_back() → Remove last element

size() → No. of elements

capacity() → Allocated memory size

empty() → Check if vector is empty.

clear() → Remove all elements

insert() → Insert at specific position

erase() → Delete element.

Advantages

- Fast random access
- Easy to use
- Automatically resizes

Disadvantages

- Slow insertion/deletion in the middle.
- May waste memory due to extra capacity.

Example:-

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v;
    v.push_back(1);
    v.push_back(2);
    cout << "size:" << v.size() << endl;
    cout << "Capacity:" << v.capacity() << endl;
}
```

Example:- To remove duplicate elements

```
#include <iostream>
#include <vector> → #include <algorithm>
using namespace std;
int main() {
    int vector<int> v = {4, 2, 1, 3, 1, 4, 3};
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    for (auto x: v)
        cout << x << " ";
}
```

(ii) List: A list is a doubly linked list in C++ STL. Unlike vector, elements are not stored in contiguous memory. Each element points to the next and previous element.

Advantages:

- (i) Fast insertion/deletion
- (ii) No need to shift elements
- (iii) Efficient for frequent modifications

Disadvantages:

- (i) No random access
- (ii) More memory usage
- (iii) Slower traversal than vector.

Important Functions:

push_front() → Insert at beginning
push_back() → Insert at end
pop_front() → Remove first element
pop_back() → Remove last element

insert() → Insert at position
erase() → Remove element
remove() → Remove specific value
reverse() → Reverse list
sort() → sort list
size() → No. of elements.

Example: #include <iostream>
#include <list>
using namespace std;
int main() {
list<int> l = {1, 2, 3};
for (int x: l)
cout << x << " ";
}

Output: 1 2 3

Example: #include <iostream>

#include <list>
using namespace std;
int main() {
list<int> l;
l.push_back(10);
l.push_front(5);
l.push_back(20);
for (int x: l)
cout << x << " ";
}

Output: 5 10 20.

Example: Erase Element

#include <iostream>
#include <list>
using namespace std;
int main() {
list<int> l = {1, 2, 3, 4};
auto it = l.begin();
advance(it, 1);
l.erase(it); // removes 2
for (int x: l)
cout << x << " ";
}

(iii) Deque in C++ STL: A deque (Double Ended Queue) is a sequence container that allows insertion and deletion from both ends (front and back).

* It is a mix of vector + queue.

* Dynamic size (like vector)

* Fast insertion/deletion at both ends

* Supports random access.

* Not strictly contiguous memory.

Important Functions

push-front(), push-back(), pop-front(), pop-back(), size(),
empty(), clear(), insert(), erase(), front(), back(),
at(index), swap(other)

Example: Adding Elements

```
#include <iostream>
#include <deque>
using namespace std;
int main() {
    deque<int> d;
    d.push_back(10);
    d.push_front(5);
    d.push_back(20);
    for (int x: d)
        cout << x << " ";
}
```

Example: Accessing Elements

```
#include <iostream>
#include <deque>
using namespace std;
int main() {
    deque<int> d = {10, 20, 30};
    cout << d[0] << endl; // 10
    cout << d.at(1) << endl; // 20
    cout << d.front() << endl; // 10
    cout << d.back() << endl; // 30
}
```

classmate

Date

Page

(iv) Array in C++ STL: A array in C++ STL ^{classmate} ^{fixed} size container that stores element in contiguous ^{fixed} memory (like normal array) but with extra STL features.

- * Fixed size
- * Fast access
- * Stored in contiguous memory
- * Safer and more convenient than C-style arrays.

Important Functions

at() → Access with bounds check

[] → Direct access

front() → First element

back() → Last element

size() → No. of elements

fill() → Fill with values

swap() → swap two arrays

empty() → check if empty.

get() → use to access the element of array.

Advantages

- * Fast access
- * STL support functions like size, at
- * Safer than normal arrays

Disadvantages

- * Fixed size (can't resize)
- * Less flexible than vector.

Examples:- Accessing Elements

classmate
Date _____
Page _____

```
#include <iostream>
#include <array>
using namespace std;
int main() {
    array<int, 3> arr = {10, 20, 30};
    cout << arr[0] << endl;
    cout << arr.at(1) << endl;
    cout << arr.front() << endl;
    cout << arr.back() << endl;
}
```

Example:- Swapping Arrays

```
#include <iostream>
#include <array>
using namespace std;
int main() {
    array<int, 3> a1 = {1, 2, 3};
    array<int, 3> a2 = {4, 5, 6};
    a1.swap(a2);
    for(int x: a1)
        cout << x << " ";
}
```

Example:- Sorting an array

```
#include <iostream>
#include <array>
#include <algorithm>
using namespace std;
int main() {
    array<int, 4> arr = {4, 1, 3, 2};
    sort(arr.begin(), arr.end());
    for(int x: arr)
        cout << x << " ";
}
```

Container Adapters:-

classmate
Date _____
Page _____

(i) Stack: A stack is a container adapter that follows the LIFO (Last In, First Out) principle.

- * Insertion and deletion only at top
- * No direct access to elements (no indexing)
- * Build on top of other containers

Common Operations

push() → Add element to top

pop() → Remove top element

top() → Access top element

empty() → check if stack is empty

size() → No. of elements

Example:-

```
#include <iostream>
#include <stack>
using namespace std;
int main() {
    stack<int> s;
    s.push(10);
    s.push(20);
    s.push(30);
    cout << s.top() << endl; // 30
    s.pop(); // removes 30
    cout << s.top(); // 20
}
```

Output: 30
20

classmate
Date _____
Page _____
Ln, first out

- ### Common operators:-

~~size() $\rightarrow$ No. of elements~~

q. rush (30);

3

30

02

classmate

- ## Applications of Queue

- (i) CPU Scheduling: Used in operating systems.
Processes are executed in the order they enter the queue.
- (ii) Prints Queue: Print jobs are stored in a queue.
- (iii) Breadth First Search (BFS): Used in graphs and trees.
- (iv) Data Buffers (I/O systems): Used in: keyboard input buffer
network data transfer.
streaming.
- (v) Call Centre Systems: Calls are handled in the order received.
- (vi) Real life waiting lines: Ticket counters
banks
hospitals

Associative Containers: Associative containers store elements in a sorted order and allow fast searching, insertion and deletion.

Types of Associative Containers

- (i) Set: Stores unique elements automatically sorted.

Example:

```
#include <iostream>
#include <set>
using namespace std;
int main() {
    set<int> s;
    s.insert(10);
    s.insert(20);
    s.insert(10); // duplicate ignored.
    for (int x: s)
        cout << x << " ";
}
```

Output: 10 20

- (ii) Multiset: Allows duplicate elements sorted order maintained.

Example:

```
#include <iostream>
#include <set>
using namespace std;
int main() {
    multiset<int> ms;
    ms.insert(10);
    ms.insert(10);
    ms.insert(20);
    for (int x: ms)
        cout << x << " ";
}
```

Output: 10 10 20.

- (iii) map: Stores key value pairs keys are unique. Sorted by key.

Example:

```
#include <iostream>
#include <map>
using namespace std;
int main() {
    map<int, string> m;
    m[1] = "Ajay";
    m[2] = "Raj";
    for (auto x: m)
        cout << x.first << " " << x.second << endl;
}
```

Output: 1 Ajay
2 Raj

- (iv) MultiMap: Stores key value pairs. Allows duplicate keys.

Example:

```
#include <iostream>
#include <map>
using namespace std;
int main() {
    multimap<int, string> mm;
    mm.insert({1, "A"});
    mm.insert({1, "B"});
    for (auto x: mm)
        cout << x.first << " " << x.second << endl;
}
```

Output: 1 A
1 B

Unordered Containers in C++ STL :-

Unordered containers store elements in ~~no particular~~ order and provide very fast operations using hashing.

Types of Unordered Containers

- (i) Unordered_set :- Stores unique elements
No order (random arrangement).

```
Ex: #include <iostream>
#include <unordered_set>
using namespace std;
int main() {
    unordered_set<int> s;
    s.insert(10);
    s.insert(20);
    s.insert(10); // duplicate ignored.
    for (int x : s)
        cout << x << " ";
}
```

Output: 20 10
(order may vary).

- (ii) Unordered_multiset :- Allows duplicate elements
No order.

```
Ex: #include <iostream>
#include <unordered_multiset>
using namespace std;
int main() {
    unordered_multiset<int> ms;
    ms.insert(10);
    ms.insert(10);
    ms.insert(20);
    for (int x : ms)
        cout << x << " ";
}
```

Output: 10 20 10
(order may vary).

- (iii) Unordered_map :- Stores key value pairs
Keys must be unique.
No sorting.

```
Ex: #include <iostream>
#include <unordered_map>
using namespace std;
int main() {
    unordered_map<int, string> m;
    m[1] = "Ajay";
    m[2] = "Raj";
    for (auto x : m)
        cout << x.first << " " << x.second << endl;
}
```

Output: 2 Raj (order may vary)
1 Ajay

- (iv) Unordered_multimap :- Stores key value pairs
Allows duplicate keys
No order.

```
Ex: #include <iostream>
#include <unordered_map>
using namespace std;
int main() {
    unordered_multimap<int, string> mm;
    mm.insert({1, "A"});
    mm.insert({1, "B"});
    for (auto x : mm)
        cout << x.first << " " << x.second << endl;
}
```

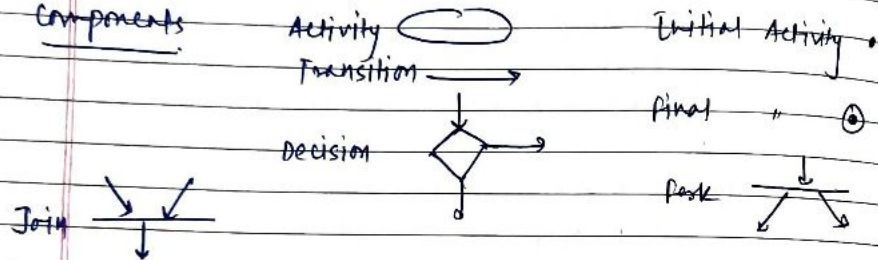
Output: 1 B (order may vary)
1 A

Activity Diagram

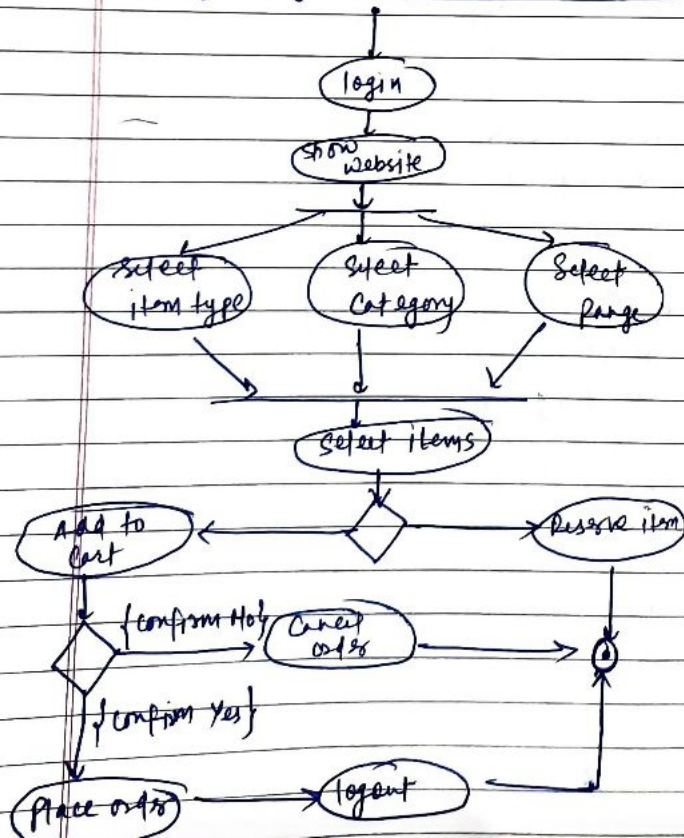
* It is a type of flow chart that explains overall flow of control.

* This flow can be sequential, branched or concurrent.
* It helps in modeling the parallel behaviour of a multithreaded system.

Components



Activity Diagram for Online Shopping.

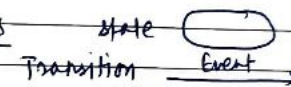


State Diagram

* A state diagram is a graph whose nodes are unique states and whose directed edges are transitions b/w states, labeled with event names.

* It gives an abstract description of the behaviour of system.
* It is used to represent the system/ different state of object and how it transitions from one state to another based on events.

Components



Guarded Transition [Guard Condition]

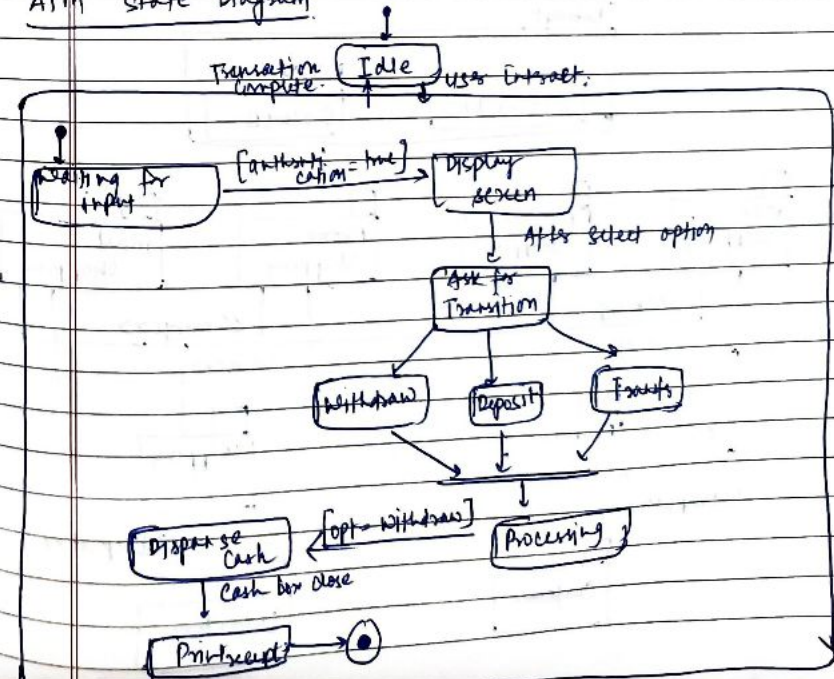
Change Event when (condition)

Time Event at (time condition)

Initial state (solid circle)

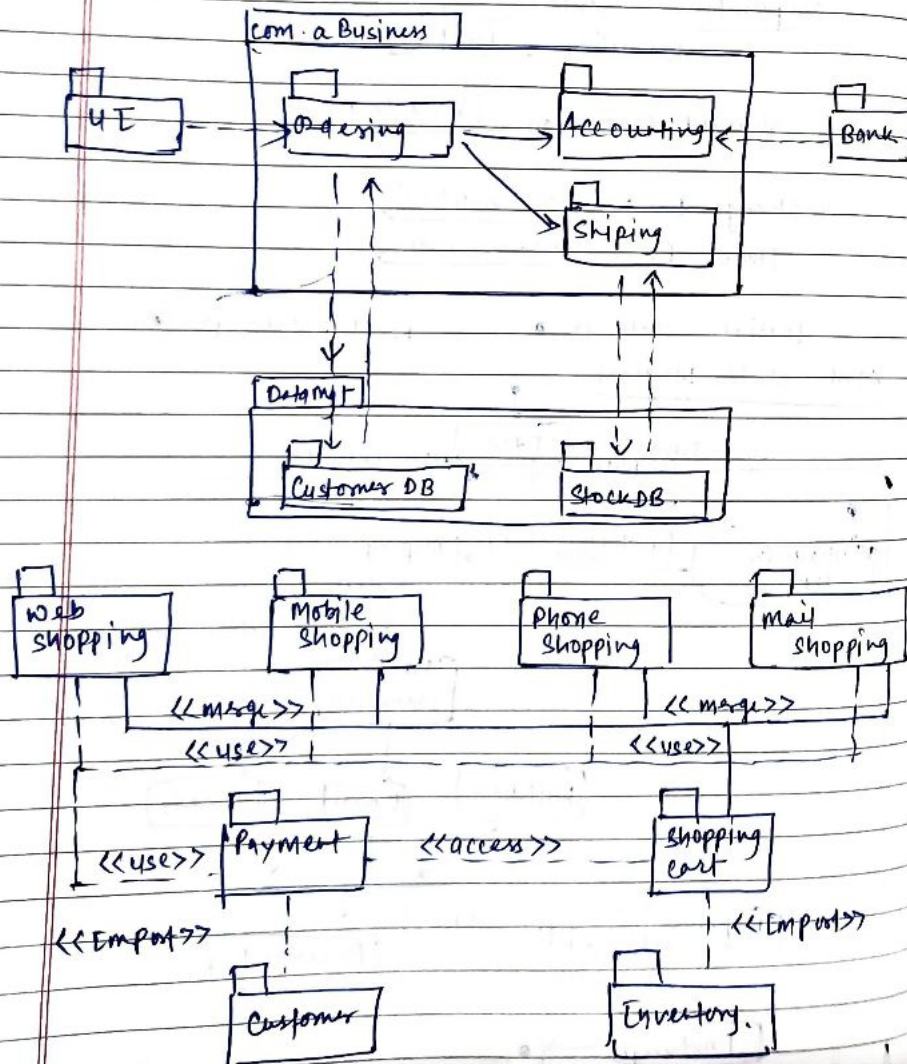
Final state (circle with a dot)

ATM State Diagram



Package Diagram:-

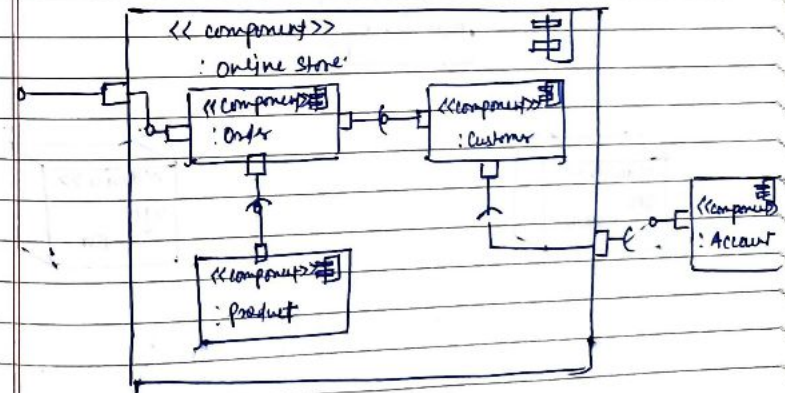
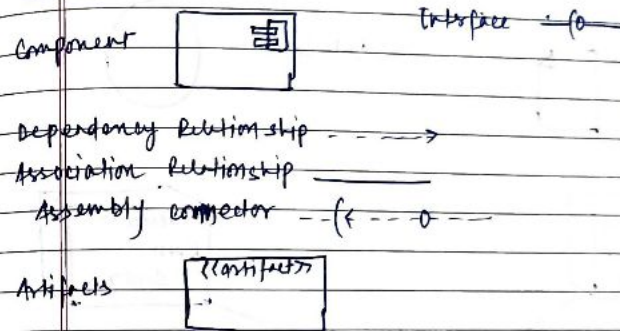
- * A package diagram is a structured UML diagram that organises system elements into packages (containers)
- * It helps developers visualise high-level architecture.
- * It includes import, access and merge relationships b/w package.
- * It serves as common language of communication b/w developer and stakeholders.



Component-Based Diagram

- * It is a kind of structural diagram in UML that shows how the components of a system are arranged and relate to one another.
- * They are widely used in system design to promote modularity, enhance understanding of system architecture.

Components:



- Component
- Assembly connector
- Artifact
- Interface
- Dependency Relationship
- Association Relationship

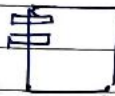
Deployment Diagram

It is a structural UML diagram that shows physical deployment of software components on hardware nodes.

Used in system architecture design, infrastructure documentation.

Components:

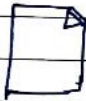
Component:



Node:



Artifact:



Interface:

